

TestPlatform

Rapport d'Execution de Campagne

testeur

Projet: ms_gestion

Version: 2.5

Environnement: ms_gestion_env

Debut: 25/06/2026 11:22 | Fin: 25/06/2026 11:24

Duree: 135.7 s

Rapport genere le 25/06/2026 11:24

ECHEC

Resume executif

Total	Succes	Echecs	Erreurs	Duree
3	0	3	0	135.7 s

Taux de reussite



Evolution vs campagne precedente: DOWN 24.0 points

Campagne precedente #19

Corrections IA: 1 test(s) corrige(s) automatiquement, 0 avec succes

Informations du contexte

Projet	ms_gestion
Description	test

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
test case add	UNIT	3	CRITICAL	IA
test add () suite test	UNIT	1	CRITICAL	IA
project create	UNIT	1	HIGH	IA

Environnement

URL Web	http://localhost:3000
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Synthese par suite

Suite	Total	Succes	Echecs	Taux
Suite UNIT	3	0	3	0%

Suite UNIT

test case add

Type	UNIT	Duree	12.7 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 4.182 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest

FAILURE

test add () suite test

Type	UNIT	Duree	19.0 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.305 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest

IA x3

FAILURE

project create

Type	UNIT	Duree	10.8 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.297 s <<< FAILURE! -- in suites.unit.ProjectServiceTest

FAILURE

Analyse IA

test case add

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 4.182 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest [ERROR] suites.unit.TestCaseServiceTest.testAddTestCase -- Time elapsed: 0.087 s <<< FAILURE! [ERROR] Failures: [ERROR] TestCaseServiceTest.testAddTestCase:101 [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 5787220878433299039\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	**Cause** : L'erreur se produit à la ligne 101 (<code>`verify`</code> sur <code>`llmService`</code>) : la méthode <code>`generateTestCode(Mockito.any(), Mockito.any(), Mockito.any())`</code> n'est pas définie avec la bonne signature dans le code réel de <code>`LlmService`</code>, ce qui entraîne une erreur de compilation ou d'exécution (généralement <code>`NullPointerException`</code> ou <code>`MockitoException`</code> silencieuse si la vérification ne correspond pas au stub). **Correction** : Vérifie la signature exacte de <code>`LlmService.generateTestCode(...)`</code> (noms, types et ordre des paramètres), puis met à jour le stub avec les bons arguments (ex. <code>`Mockito.any(String.class)`</code>, <code>`Mockito.any(TestCase.class)`</code>, <code>`Mockito.any(Project.class)`</code>), et remplace <code>`Mockito.any()`</code> non typé par des <code>`any(type)`</code> explicites pour garantir la correspondance. **Conseil** : Toujours utiliser <code>`Mockito.verify(mock, Mockito.times(1)).methode(...)`</code> avec les mêmes arguments **et types** que dans le stub ou l'appel réel, car Mockito effectue une vérification stricte basée sur la signature complète de la méthode.

test add () suite test

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.305 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest [ERROR] suites.unit.TestSuiteServiceUnitTest.test_add_happyPath -- Time elapsed: 0.050 s <<< FAILURE! java.lang.AssertionError: expected [TestProject] but found [null] [ERROR] Failures: [ERROR] TestSuiteServiceUnitTest.test_add_happyPath:95 expected [TestProject] but found [null] [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-</pre>
----------------------	---

	<pre> plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 5787220878433299039\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException </pre>
Analyse IA	**Cause** : La méthode <code>TestSuiteService.add()</code> retourne <code>null</code> au lieu d'un <code>TestSuiteResponse</code>, car la logique métier ne construit pas ou ne renvoie pas la réponse attendue (probablement un bug dans <code>TestSuiteService.add()</code> ou une erreur de mappage non gérée). **Correction** : Vérifiez l'implémentation de <code>TestSuiteService.add()</code> – assurez-vous qu'elle instancie et retourne un <code>TestSuiteResponse</code> avec les bonnes valeurs (ex. <code>response.setName(request.getName())</code>), ou corrigez le mock de <code>testSuiteRepository.save()</code> pour qu'il simule correctement le comportement (ex. <code>thenReturn(saved)</code> doit bien être utilisé, et la méthode <code>add()</code> doit mapper <code>saved</code> vers une réponse). **Conseil** : Utilisez <code>@Spy</code> ou des assertions intermédiaires (<code>assertEquals(response, null)</code> ou <code>assertThat(response).isNotNull()</code>) dans les tests pour détecter tôt les erreurs de mappage ou de logique métier.

project create

Erreurs extraites	<pre> [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.297 s <<< FAILURE! -- in suites.unit.ProjectServiceTest [ERROR] suites.unit.ProjectServiceTest.test_create_happy_path -- Time elapsed: 0.048 s <<< FAILURE! [ERROR] Failures: [ERROR] ProjectServiceTest.test_create_happy_path:44 Runtime Utilisateur non authentifi [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 5787220878433299039\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException </pre>
Analyse IA	**Cause** : L'erreur « Utilisateur non authentifié » provient de la méthode <code>ProjectService.create()</code> qui tente d'accéder à un utilisateur courant via une entité Spring Security (ex. <code>@AuthenticationPrincipal</code> ou <code>SecurityContextHolder</code>) non simulée

dans le test unitaire.

****Correction**** :

1. Ajouter `@ExtendWith(MockitoExtension.class)` ou équivalent pour activer correctement les mocks (si non utilisé, vérifier l'initialisation via `@ExtendWith` ou `MockitoAnnotations.openMocks(this)`).
2. Dans le test, simuler le contexte de sécurité :
``java
Authentication auth = MockAuthenticationBuilder.mockAuthentication("test-user");
SecurityContextHolder.getContext().setAuthentication(auth);
``
(ou utiliser `@WithMockUser` si Spring Security est bien configuré en mode test, via `spring-security-test`).
3. Vérifier que `ProjectService.create()` n'appelle pas de méthode bloquante non mockée (ex. `userService.getCurrentUser()`).

****Conseil**** : En unité, isole le service en mockant ****toutes**** les dépendances externes (y compris Spring Security) ou utilisez `@WithMockUser` pour simplifier la simulation d'authentification.

Corrections automatiques par IA

L'IA a detecte des erreurs dans certains scripts generes et les a corriges automatiquement.

test add () suite test

3 correction(s)

FAILURE

```
=== Tentative 1/3 ===
Erreur détectée: [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.973 s <<<
FAILURE! -- in suites.unit.TestSuiteServiceUnitTest
[ERROR] suites.unit.TestSuiteServiceUnitTest.test_add_happyPath -- Time elapsed: 0.055 s <<< FAILURE!
java.lang.AssertionError: expected [TestProject] but found [null]
[ERROR] Failures:
[ERROR]   TestSuiteServiceUnitTest.test_add_happyPath:89 expected [TestProject] but found [null]
[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
[E
Résultat: Script corrigé (3836 chars), relance Maven...
Résultat: Encore en échec (exit=1).

=== Tentative 2/3 ===
Erreur détectée: [ERROR] COMPILATION ERROR :
[ERROR] /C:/tmp/ms-execution/exec-5787220878433299039/Backend/ms_gestion/src/test/java/suites/unit/TestSuiteServiceUnitTest.java:[74,46]
constructor TestSuiteResponse in class com.pfe.platform.ms_gestion.dto.response.TestSuiteResponse cannot be
applied to given types;
[ERROR] /C:/tmp/ms-execution/exec-5787220878433299039/Backend/ms_gestion/src/test/java/suites/unit/TestSuiteServiceUnitTest.java:[77,25]
cannot find symbol
[ERROR] /C:/tmp/ms-execution/exec-578722087843
Résultat: Script corrigé (3809 chars), relance Maven...
Résultat: Encore en échec (exit=1).

=== Tentative 3/3 ===
Erreur détectée: [ERROR] COMPILATION ERROR :
[ERROR] /C:/tmp/ms-execution/exec-5787220878433299039/Backend/ms_gestion/src/test/java/suites/unit/TestSuiteServiceUnitTest.java:[74,46]
TestSuiteResponse(java.lang.Long,java.lang.String,java.lang.String,java.lang.String,java.lang.String,java.la
ng.String,java.lang.String,java.time.LocalDateTime) is not public in
com.pfe.platform.ms_gestion.dto.response.TestSuiteResponse; cannot be accessed from outside package
[ERROR] /C:/tmp/ms-execution/exec-5787220878433299039/Bac
Résultat: Script corrigé (3488 chars), relance Maven...
Résultat: Encore en échec (exit=1).
```


Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
test add () suite test	Suite UNIT	19.0 s
test case add	Suite UNIT	12.7 s
project create	Suite UNIT	10.8 s

Temps d'execution par type

Type	Duree moyenne
UNIT	14.2 s

Tests instables

Aucun test instable detecte

Recommandations

- test case add:

****Cause**** :

L'erreur se produit à la ligne 101 (``verify`` sur ``llmService``) : la méthode ``generateTestCode(Mockito.any(), Mockito.any(), Mockito.any())`` n'est pas définie avec la bonne signature dans le code réel de ``LlmService``, ce qui entraîne une erreur de compilation ou d'exécution (généralement ``NullPointerException`` ou ``MockitoException`` silencieuse si la vérification ne correspond pas au stub).

****Correction**** :

Vérifie la signature exacte de ``LlmService.generateTestCode(...)`` (noms, types et ordre des paramètres), puis met à jour le stub avec les bons arguments (ex. ``Mockito.any(String.class)``, ``Mockito.any(TestCase.class)``, ``Mockito.any(Project.class)``), et remplace ``Mockito.any()`` non typé par des ``any(type)`` explicites pour garantir la correspondance.

****Conseil**** :

Toujours utiliser ``Mockito.verify(mock, Mockito.times(1)).methode(...)`` avec les mêmes arguments ****et types**** que dans le stub ou l'appel réel, car Mockito effectue une vérification stricte basée sur la signature complète de la méthode.

- test add () suite test:

****Cause**** : La méthode ``TestSuiteService.add()`` retourne ``null`` au lieu d'un ``TestSuiteResponse``, car la logique métier ne construit pas ou ne renvoie pas la réponse attendue (probablement un bug dans ``TestSuiteService.add()`` ou une erreur de mappage non gérée).

****Correction**** : Vérifiez l'implémentation de ``TestSuiteService.add()`` — assurez-vous qu'elle instancie et retourne un ``TestSuiteResponse`` avec les bonnes valeurs (ex. ``response.setName(request.getName())``), ou corrigez le mock de ``testSuiteRepository.save()`` pour qu'il simule correctement le comportement (ex. ``thenReturn(saved)`` doit bien être utilisé, et la méthode ``add()`` doit mapper ``saved`` vers une réponse).

****Conseil**** : Utilisez ``@Spy`` ou des assertions intermédiaires (``assertEquals(response, null)`` ou ``assertThat(response).isNull()``) dans les tests pour détecter tôt les erreurs de mappage ou de logique métier.

- project create:

****Cause**** : L'erreur « Utilisateur non authentifié » provient de la méthode ``ProjectService.create()`` qui tente d'accéder à un utilisateur courant via une entité Spring Security (ex. ``@AuthenticationPrincipal`` ou ``SecurityContextHolder``) non simulée dans le test unitaire.

****Correction**** :

1. Ajouter ``@ExtendWith(MockitoExtension.class)`` ou équivalent pour activer correctement les mocks (si non utilisé, vérifier l'initialisation via ``@ExtendWith`` ou ``MockitoAnnotations.openMocks(this)``).

2. Dans le test, simuler le contexte de sécurité :

```
```java
```

```
Authentication auth = MockAuthenticationBuilder.mockAuthentication("test-user");
```

```
SecurityContextHolder.getContext().setAuthentication(auth);
```

```
```
```

(ou utiliser ``@WithMockUser`` si Spring Security est bien configuré en mode test, via ``spring-security-test``).

3. Vérifier que ``ProjectService.create()`` n'appelle pas de méthode bloquante non mockée (ex.

```
`userService.getCurrentUser()`).
```

****Conseil**** : En unité, isole le service en mockant ****toutes**** les dépendances externes (y compris Spring Security) ou utilisez `@WithMockUser` pour simplifier la simulation d'authentification.

Annexe - Scripts generes par IA

test case add

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
        member.setProject(project);

        TestSuite suite = new TestSuite();
```

```

suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));

Mockito.when(llmService.generateTestCode(
Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");

Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
TestCase tc = inv.getArgument(0);
tc.setSuite(suite);
return tc;
});

TestCaseResponse response = testCaseService.add(suiteId, request);

Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
// verify
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any());
}
}

```

test add () suite test

```

package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestSuiteRequest;
import com.pfe.platform.ms_gestion.dto.response.TestSuiteResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.TestSuiteService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Optional;

public class TestSuiteServiceUnitTest {

```

```
@Mock private TestSuiteRepository testSuiteRepository;
@Mock private ProjectRepository projectRepository;
@Mock private ProjectMemberRepository projectMemberRepository;

@InjectMocks
private TestSuiteService testSuiteService; // pas de new()

private AutoCloseable mocks;
private MockedStatic<SecurityUtils> mockedSecurity;

@BeforeMethod
public void setUp() {
    mocks = MockitoAnnotations.openMocks(this);
    mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
    mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
}

@AfterMethod
public void tearDown() throws Exception {
    mockedSecurity.close();
    mocks.close();
}

@Test
public void test_add_happyPath() {
    // Arrange
    Long projectId = 1L;

    Project project = new Project();
    project.setId(projectId);

    ProjectMember member = new ProjectMember();
    member.setUserId(1L);
    member.setRole(ProjectMember.Role.ADMIN);
    member.setProject(project);

    TestSuite saved = new TestSuite();
    saved.setId(10L);
    saved.setProject(project);

    // Mock repositories
    Mockito.when(projectRepository.findById(projectId))
        .thenReturn(Optional.of(project));
    Mockito.when(projectMemberRepository.findByProjectIdAndUserId(
        Mockito.eq(projectId), Mockito.eq(1L)))
        .thenReturn(Optional.of(member));
    Mockito.when(testSuiteRepository.save(Mockito.any()))
        .thenReturn(saved);

    // Request avec les test data générés
    CreateTestSuiteRequest request = new CreateTestSuiteRequest();
    request.setName("TestProject");
    request.setType("UNIT");
    request.setGitRepoUrl("https://github.com/testproject.git");
    request.setGitBranch("master");
    request.setModulePath("/src/main/java/com/example/test");

    // Act
    TestSuiteResponse response = testSuiteService.add(projectId, request);

    // Assert
    Assert.assertNotNull(response);
    Assert.assertEquals(response.getName(), "TestProject");
    Mockito.verify(testSuiteRepository, Mockito.times(1)).save(Mockito.any());
    Mockito.verify(projectRepository, Mockito.times(1)).findById(projectId);
}
```

project create

```
package suites.unit; //

import com.pfe.platform.ms_gestion.service.ProjectService;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.dto.request.CreateProjectRequest;
import com.pfe.platform.ms_gestion.dto.response.ProjectResponse;
import org.mockito.*;
import org.testng.annotations.*;
import static org.testng.Assert.*;
import static org.mockito.Mockito.*;

public class ProjectServiceTest {

    private AutoCloseable mocks; //

    @Mock private ProjectRepository projectRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @InjectMocks private ProjectService sut;

    @BeforeMethod
    public void setUp() throws Exception {
        mocks = MockitoAnnotations.openMocks(this); //
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mocks.close(); //
    }

    @Test
    public void test_create_happy_path() {
        // given
        CreateProjectRequest request = new CreateProjectRequest();
        request.setName("test");

        Project saved = new Project();
        saved.setId(1L);
        saved.setName("test"); // entité réelle, pas null
        when(projectRepository.save(any())).thenReturn(saved);

        // when
        ProjectResponse result = sut.create(request);

        // then
        assertNotNull(result);
        assertEquals(result.getName(), "test");
        verify(projectRepository, times(1)).save(any());
    }
}
```